Benjamin Escobar

SWDV 691

11/7/2026

ShredTracker: Service Layer Re-Design

Changes from old design

The service layer now includes five new endpoints for creating, viewing, updating and deleting saved workout routines such as a stretch features. Request, response, and error examples are added and the diagram was updated to identify MVP and stretch feature connections. The existing MVP endpoints will remain the same with the exception of small corrections to the weight goal response goal and remaining weight calculation.

Introduction

ShredTracker will use Node.js/Express REST API between its React interface and MySQL database. The service will authenticate user input, enforce ownership, save records, and calculate weight goal progress and workout comparisons. The API will be served under /api on the same origin as the website and the actual Heroku hostname will be assigned during deployment.

Request and Response

Requests and Responses will use JSON except for successful deletions and logouts, these will return 204 (No Content) with no response body. GET will read data, POST creates records or sessions, PUT updates records, and DELETE removes them. Login and registration create a secure session cookie, while all other endpoints require the session to identify the user. The service will check the request's origin before changing data and never returns passwords hashes or session values in JSON. Missing sessions will return 401, missing records will return 404, and ID's must be positive integers.

Shared Error Format

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Repetitions must be a positive whole number.",
    "field": "sets[0].repetitions"
  }
}
```

Error codes will include 400 for invalid input, 401 for login issues, 403 for blocked request origin, 404 for missing records, 409 for duplicate entries, and 500 for server errors. The field property will identify the invalid input and is left out when it does not apply. Dates will use YYYY-MM-DD format and weights are numbers in pounds. Empty lists will appear as [] and notes will be null when left blank.

POST /api/users

Creates a suer account using a name, valid email, and password between 8-128 characters. The server will store password hash, remove extra spaces from emails, and convert the email to lowercase.
Request:

```
{
  "name": "Morgan Winnebaker",
  "email": "morgan@example.com",
  "password": "DemoPassword42!"
}
```

Response 201:

```
{
  "user_id": 1,
  "name": "Morgan Winnebaker",
  "email": "morgan@example.com"
}
```

Error Response: 409 Conflict

```
{
  "error": {
    "code": "EMAIL_EXISTS",
    "message": "An account with this email already exists."
     }
}
```

POST /api/session

Signing in an existing user by checking their email and password against the password hash. A successful login will create a session without changing their fitness records.

Request:

```
{
 "email": "morgan@example.com",
 "password": "DemoPassword42!"
}
```

Response: 200 OK

```
{
 "user_id": 1,
 "name": "Morgan Winnebaker",
 "email": "morgan@example.com"
}
```

Error Response: 401 Unauthorized (Returned when email or password are incorrect)

```
{
 "error": {
  "code": "INVALID_CREDENTIALS",
  "message": "Email or password is incorrect."
 }
}
```

GET /api/session

Checks whether the user is signed in and returns their account information. No database records are changed.

Request:

GET /api/session

Response: 200 OK

```
{
  "user_id": 1,
  "name": "Morgan Winnebaker",
```

```
  "email": "morgan@example.com"
}
```

Error Response: 401 Unauthorized (Returned when session cookie is missing or expired)

```
{
  "error": {
    "code": "UNAUTHENTICATED",
    "message": "Please log in."
  }
}
```

DELETE /api/session

Logs the user out by ending their session and clearing the session cookie. Their account and fitness records are saved.

Request:

DELETE /api/session

Response: 204 No Content

Logout was successful, no response body is returned.

GET /api/exercises

Returns the exercise list and weight entry instructions so user can select an exercise when adding or editing a workout. No records are changed.

Request:

GET /api/exercises

Response: 200 OK

{

"exercises": [

{

"exercise_id": 1,

"name": "Bench press",

```
    "load_instructions": "Enter total weight including the bar."
   }
 ]
}
```


GET /api/workouts

Returns the signed in user's complete workout history so users can review past sessions. Workouts appear newest first, sorted by date and then workout ID.

Request:

GET /api/workouts

Response: 200 OK


```
{
  "workouts": [
   {
    "workout_id": 12,
    "workout_date": "2026-10-30",
    "notes": "Morning workout"
   }
 ]
}
```


POST /api/workouts

Saves user's workout and its sets together. If any of the parts fail, then nothing gets saved. All fields below are required except notes and workout must include a valid set.

Request:

Post /api/workouts


```
{
  "workout_date": "2026-10-30",
```

```
 "notes": "Morning workout",
 "sets": [
  {
   "exercise_id": 1,
   "set_number": 1,
   "weight_lb": 100,
   "repetitions": 10
  }
 ]
}
```


Response: 201 Created


```
{
 "workout_id": 12,
 "workout_date": "2026-10-30",
 "notes": "Morning workout",
 "sets": [
  {
   "set_id": 31,
   "exercise_id": 1,
   "set_number": 1,
   "weight_lb": 100,
   "repetitions": 10
  }
 ]
}
```


Error Response: 400 Bad Request (Returns when repetitions are entered as 0)


```
{
 "error": {
  "code": "VALIDATION_ERROR",
  "message": "Repetitions must be a positive whole number.",
  "field": "sets[0].repetitions"
 }
}
```


GET /api/workouts/12

User opens work out details and returns workout 12 to view or edit. It also compares this workout with its previous one by using the date and workout ID.

Request:

GET /api/workouts/12

Response: 200 OK


```
{
 "workout_id": 12,
 "workout_date": "2026-10-30",
 "notes": null,
 "sets": [
  {
   "set_id": 31,
   "exercise_id": 1,
   "set_number": 1,
   "weight_lb": 100,
   "repetitions": 10
  }
 ],
 "comparisons": [
  {
   "exercise_id": 1,
   "exercise_name": "Bench press",
   "volume_lb": 1000,
   "previous_volume_lb": 900,
   "change_percent": 11.11,
   "best_weight_lb": 100,
   "is_new_record": false
  }
 ]
}
```


Error Response: 404 Not found (Returned when the workout doesn't exist or belongs to another user)


```
{
 "error": {
  "code": "NOT_FOUND",
  "message": "Workout not found."
 }
}
```

PUT /api/workouts/12

User can update workout date, notes and full list of sets. Sets that are left out of the request are removed and server assigns new set ID's. All changes are saved together, but if an error happens the original workout stays unchanged.

Request:


```
{
  "workout_date": "2026-10-30",
  "notes": "Corrected load",
  "sets": [
    {
      "exercise_id": 1,
      "set_number": 1,
      "weight_lb": 105,
      "repetitions": 10
    }
  ]
}
```


Response: 200 OK


```
{
  "workout_id": 12,
  "workout_date": "2026-10-30",
  "notes": "Corrected load",
  "sets": [
    {
      "set_id": 32,
```

```
    "exercise_id": 1,
    "set_number": 1,
    "weight_lb": 105,
    "repetitions": 10
   }
 ]
}
```


Error Response: 400 Bad Request (Returned when the request contains an empty list set, ex: [].)


```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Add at least one set.",
    "field": "sets"
  }
}
```


DELETE /api/workouts/12

Deletes a user's workout and its sets after they confirm the action. The interface refreshes with the users workout history and any displayed stats.

Request:

DELETE /api/workouts/12

Response: 204 No Content

Error Response: 404 Not Found (Returned when the workout does not exist or belongs to another user.)


```
{
```

```
 "error": {
  "code": "NOT_FOUND",
  "message": "Workout not found."
 }
}
```


GET /api/weight-entries

Returns a user's complete weight history from oldest to newest. The readings are used for the line chart and editing existing entries.

Request:

GET /api/weight-entries

Response: 200 OK


```
{
 "weight_entries": [
  {
   "weight_entry_id": 21,
   "entry_date": "2026-10-01",
   "weight_lb": 150
  },
  {
   "weight_entry_id": 22,
   "entry_date": "2026-10-30",
   "weight_lb": 144
  }
 ]
}
```


Workout Validation and Calculation Rules

For validation, exercises must exist. Set numbers and repetitions must be positive integers and no duplicate set numbers for the same exercise in a workout. Weights allow 0-9999.99 pounds and two decimal places. Notes are allowed 500 characters. Volume totals weight x reps across sets. Percentage change is (current – previous volume) / previous volume x 100, rounded two decimals or null if previous volume is missing or zero. The highest load is

the personal best. First sessions and ties are not new records. Results are calculated from saved workouts.

POST /api/weight-entries

Saves a user's daily weight reading. Requires a valid date and weight greater than zero, up to 9999.99lbs with two decimal place.

Request:

POST /api/weight-entries


```
{
 "entry_date": "2026-10-30",
 "weight_lb": 144
}
```


Response: 201 Created


```
{
 "weight_entry_id": 22,
 "entry_date": "2026-10-30",
 "weight_lb": 144
}
```


Error Response: 409 Conflict (Returned when the user already has a reading for that date.)


```
{
 "error": {
  "code": "DATE_EXISTS",
  "message": "Edit the existing reading for this date.",
  "field": "entry_date"
 }
}
```


PUT /api/weight-entries/22

User's update existing weight reading. The date and weight must be included. After saving, the interface refreshes the chard and goal progress.

Request:


```
{
```

```
 "entry_date": "2026-10-30",
 "weight_lb": 143.5
}
```


Response: 200 OK


```
{
 "weight_entry_id": 22,
 "entry_date": "2026-10-30",
 "weight_lb": 143.5
}
```


Error Response: 400 Bad Request (Returned when the weight is entered as -5.)


```
{
 "error": {
  "code": "VALIDATION_ERROR",
  "message": "Weight must be greater than zero.",
  "field": "weight_lb"
 }
}
```


If date already used in another reading, it'll return 409 DATE_EXISTS. If missing or belongs to another user returns 404 NOT_FOUND with message saying weight entry not found.

DELETE /api/weight-entries/22

Deletes user's weight reading after confirming the action. The interface then refreshes the chart and goal progress.

Request:

DELETE /api/weight-entries/22

Response: 204 No Content

Error Response: 404 Not Found (Returned when the reading does not exist or belongs to another user)


```
{
 "error": {
  "code": "NOT_FOUND",
  "message": "Weight entry not found."
 }
```

}

GET /api/weight-goal

Returns User's goal and progress using their latest weight reading on or after the goal's start date. This information fills the goal form and weight progress page.

Request:

GET /api/weight-goal

Response: 200 OK


```
{
  "goal": {
    "goal_id": 5,
    "start_date": "2026-10-01",
    "starting_weight_lb": 150,
    "target_weight_lb": 120
  },
  "progress": {
    "current_weight_lb": 144,
    "weight_lost_lb": 6,
    "remaining_lb": 24,
    "progress_percent": 20
  }
}
```


If no goals exist returns 200 OK with:


```
{
  "goal": null,
  "progress": null
```

```
}
```

If goal exists but has no weight reading on or after the start date, then it returns the goal with progress: null.

PUT /api/weight-goal

Creates or updates a user's weight-loss goal. All three fields are required. Weight must be positive and target must be below start weight.

Request:

```
{
  "start_date": "2026-10-01",
  "starting_weight_lb": 150,
  "target_weight_lb": 120
}
```

Response: 201 Created for new goal or 200 for updated goal

```
{
  "goal_id": 5,
  "start_date": "2026-10-01",
  "starting_weight_lb": 150,
  "target_weight_lb": 120
}
```

Error Response: 400 Bad Request (Returned when target weight is above starting weight.)

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Target weight must be below starting weight.",
    "field": "target_weight_lb"
  }
}
```

GET /api/dashboard

Returns the signed-in user's summary cards, weight chart and goal progress. Results are calculated from saved records without storing totals. Bench press records are identified by the configured exercise ID.

Request:

GET /api/dashboard

Response: 200 OK


```
{
  "latest_bench_press": {
    "workout_id": 12,
    "volume_lb": 1000,
    "previous_volume_lb": 900,
    "change_percent": 11.11
  },
  "weight_change_since_start_lb": -6,
  "weight_points": [
    {
      "date": "2026-10-01",
      "weight_lb": 150
    },
    {
      "date": "2026-10-30",
      "weight_lb": 144
    }
  ],
  "goal_progress": {
    "current_weight_lb": 144,
    "weight_lost_lb": 6,
    "remaining_lb": 24,
    "progress_percent": 20
```

}

}

The chart will include all weight readings in date order. Weight change equals the latest reading minus the earliest: one reading returns 0, and no readings returns null. Missing bench press history returns latest_bench_press: null. A missing goal or weight reading on or after the start daye returns goal_progress: null.

Goal Calculations and Refresh Behavior

Uses the latest weight recorded on or after the goal's start date. Weight lost is starting weight – current weight. Pounds remaining is the current weight – target weight, with a minimum of zero. Goal progress is weight lost / total pounds to lose * 100 and rounded two decimals with percentage between 0% and 100%. Refresh related page after save or deleting information and refresh the dashboard when it opens so it shows current results.

Stretch Feature: Saved Routines

User saves reusable workout plans. WorkoutRoutine stores the owner and name, RoutineExcerise stores the exercises and planned values. All endpoints require a session and only access the signed-in user's records. Names require 1-100 characters, at least one existing exercise is required. Order, sets, and reps must be positive integers. Weight allows 0-9999.99 lbs and two decimal places. All request fields are below required.

GET /api/routines

Lists the user's routines by name then ID.

Request: GET /api/routines (no body)
200 OK: {"routines":[{"routine_id":7,"name":"Upper Body"}]}
Empty list: {"routines":[]}

POST /api/routines

Creates a routine and its entries together. IF any fails, nothing will be saved.

Request: POST /api/routines
{"name":"Upper Body","exercises":[
{"exercise_id":1,"exercise_order":1,"planned_sets":3,
"planned_repetitions":10,"planned_weight_lb":100}]}

201 Created
{"routine_id":7,"name":"Upper Body","exercises":[
{"routine_exercise_id":41,"exercise_id":1,"exercise_order":1,
"planned_sets":3,"planned_repetitions":10,
"planned_weight_lb":100}]}

400 if exercises is []:
{"error":{"code":"VALIDATION_ERROR",
"message":"Add at least one exercise.","field":"exercises"}}

GET /api/routines/7

Reads a user's owned routine with exercise detail, ordered by exercise_order for editing or loading a workout.

Request: GET /api/routines/7 (no body)
200 OK
{"routine_id":7,"name":"Upper Body","exercises":[
{"routine_exercise_id":41,"exercise_id":1,
"exercise_name":"Bench press",
"load_instructions":"Enter total weight including the bar.",
"exercise_order":1,"planned_sets":3,
"planned_repetitions":10,"planned_weight_lb":100}]}

PUT /api/routines/7

Replaces the routine name and full exercise list. Entries are removed and new entry IDs are assigned. A failed update will leave the original routine unchanged.

Request: PUT /api/routines/7
{"name":"Upper Body","exercises":[
{"exercise_id":1,"exercise_order":1,"planned_sets":3,
"planned_repetitions":8,"planned_weight_lb":105}]}

200 OK
{"routine_id":7,"name":"Upper Body","exercises":[
{"routine_exercise_id":42,"exercise_id":1,"exercise_order":1,
"planned_sets":3,"planned_repetitions":8,
"planned_weight_lb":105}]}

400 if planned_sets is 0:
{"error":{"code":"VALIDATION_ERROR",
"message":"Planned sets must be a positive whole number.",
"field":"exercises[0].planned_sets"}}

DELETE /api/routines/7

After confirmation, deletes the owned routine and entries. The existing workouts remain unchanged.

Request: DELETE /api/routines/7 (no body)
Response: 204 No Content (no body)

Shared Errors

All routine endpoints return 401 for a misting or expired session. GET, PUT and DELETE of one routine will return 401 if it missing or belongs to another user.

401: {"error":{"code":"UNAUTHENTICATED",
"message":"Please log in."}}

404: {"error":{"code":"NOT_FOUND",
"message":"Routine not found."}}

Using a routine

Selecting a routine will fill the workout form with its planned exercises and sets. Users can adjust the values and save through the POST /api/workout endpoint. Changing or deleting a routine does not affect the past workouts.

Page to Endpoint Diagram

